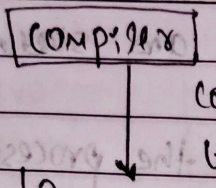
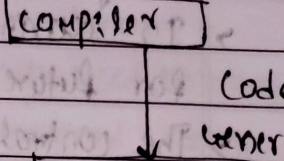
D D M M Y Y Y Y

Assignment -01

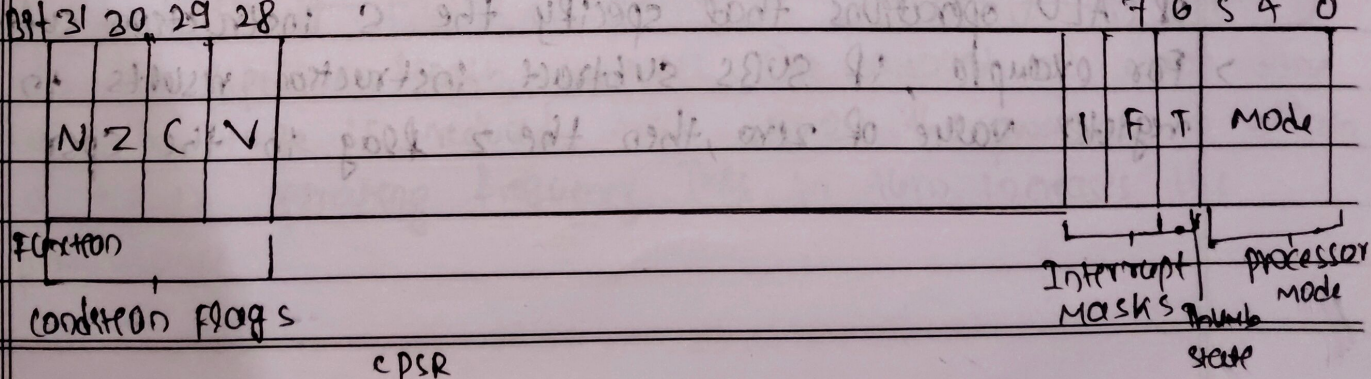
01) Compare RISC and CISC processors

→

RISC	CISC
> Greater complexity 	>  complexity
> Emphasizes on compiler complexity.	> Emphasizes on processor complexity.
> Simple but powerful Instructions.	> Instructions are more complicated.
> Executes instruction in single cycle.	> Takes many cycle to execute.
> Instructions are fixed length.	> Instructions are of variable length
> Have large set of general purpose registers.	> Have limited set of general purpose registers.
> Any register can contain either data or an address.	> Dedicated registers for specific purpose.
> Separate load and store instructions transfer data b/w the register & external memory.	> MOV instructions can be used to transfer b/w register and memory.

02) Describe the current program status register of ARM with the help of bit layout diagram.

→ Fields ← flags ← status ← Extension ← control ←



CPSR

D D M M Y Y Y Y

> The cpsr is divided into four fields, each 8 bits wide: flags, status, extension and control.
> In current designs the extension and status fields are reserved for future use.
> The control field ~~controls~~ contains the processor mode, state and interrupts mask bits.
> The flag field contains the condition flags.
> The following table gives the bit patterns that represent each of the processor modes in the cpsr.

Mode	Mode [4:0]
Abort	10111
Fast Interrupt request	10001
Interrupt request	10010
Supervisior	10011
System	11111
Undefined	11011
User	10000

> When cpsr bit 5, T=1, then the processor is in Thumb state. When T=0, the processor is in ARM state.
> The cpsr has two interrupt mask bits, 7 and 6 (I and F) which control the masking Interrupt request (IRQ) and Fast Interrupt Request (FIR).
> condition flags are updated by comparisons and the result of ALU operations that specify the S instruction suffix.
> For example, if SUBS subtract instruction results in a register value of zero, then the Z flag in the cpsr is set.

D D M M Y Y Y Y

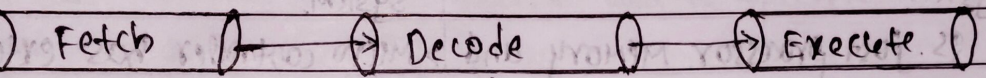
Conditional Flags

Flag	Flag Name	Set when
N	Negative	Bit 31 of the result is a binary 1
Z	Zero	The result is zero, frequently used to indicate equality
C	Carry	The result causes an unsigned carry
V	Overflow	The result causes a signed overflow.

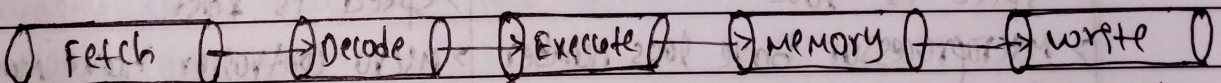
06] Define Pipeline in ARM? Illustrate with an example.

→ • Pipeline is the mechanism to speed up execution by fetching the next instruction while other instructions are being decoded and executed.

• ARM7 three-stage Pipeline



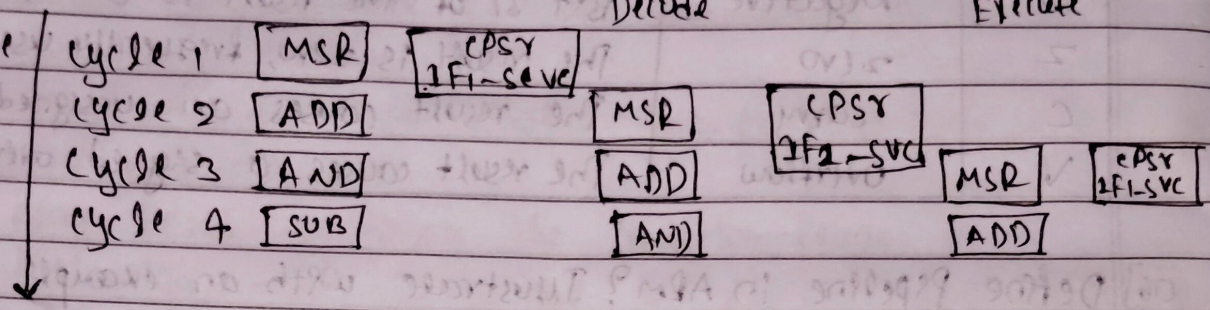
• Fetch loads an instruction from memory.
• Decode identifies the instruction to be executed.
• Execute processes the instruction and writes the result back to a register.
• Each instruction cycle takes a single cycle to complete after the pipeline is filled.
• The pipeline design for each ARM family differs. For example, the ARM9 core increases the pipeline length to five stages as shown below.



• As the pipeline length increases the amount of work done at each stage is reduced, which allows the processor to attain a higher operating frequency. This in turn increases the performance.

D D M M Y Y Y Y

- For example, an ARM7 pipeline has executed an instruction only when the fourth instruction is fetched.



07] Compare and contrast Microprocessor & Microcontroller.

Microprocessor	Microcontroller
> It is a heart of computer system.	> It is heart of embeded system.
> It is just processor. Memory and I/O components have to be connected externally.	> Micro controller has external processor along with internal memory and i/o components.
> cannot be used in compact systems and hence ineffecient.	> can be used in compact systems and hence it is an effecient tecnique.
> Cost of the entire system increases.	> Cost of entire system is low.
> Due to external components, the entire power consumption is high.	> Since external components are low, total power consumption is low.
> Do not have power saving features.	> Having power saving modes like idle modes.
> speed is low	> speed is fast.
> Microprocessor have less number of registers, hence more operations are memory based.	> Micro controller have more number of registers, hence the programs are easily to write.

D D M M Y Y Y Y

08] Explain with neat diagram of an ARM based embeded device.

→ • An embeded system requires software to drive it.
• Each software components in the stack uses a higher level of abstraction to seperate the code from the hardware device.

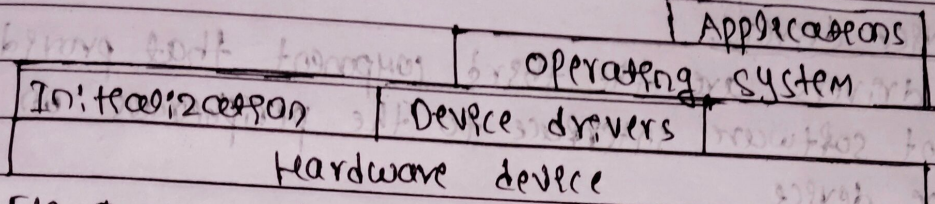


Fig: Typical software components required to control an embeded system

Initialization (BOOT) code

• Initialization code takes the processor from the rest state to a state where the OS can run.
• First code executed on the board and is specific to a particular target or group of targets.
• Handles a number of administative tasks prior to handing control over to an OS.
• We can group these different tasks into three phases.
• Initial hardware configuration involves setting up the target platform so it can boot an image.
• Diagnostics: The primary purpose of diagnostic code is fault identification and isolation.
• Booting: Involves loading an image and handing control over the image. Loading an image involves copying an entry program including code and data into RAM.

The operating system

• It organizes the system resources: the peripherals, memory and processing time.
• ARM processors support over 50 operating systems.
• We can divide OS into 2 main categories: real time OS (RTOs) & platform OS.

- RTOSs provide guranted response time to events. Systems running an RTOS generally do not have secondary storage.
- Platform OS require memory management unit to manage large, non-real time applications and tends to have secondary storage.

The device drivers:

- Device drivers are the third component that provides a consistent software interface to the peripherals on the hardware device.

og] Describe the different processor modes provided by ARM7.

→ Each processor mode is either privileged or nonprivileged.

- A privileged mode allows read-write access to the cprs.
- A nonprivileged mode only allows read access to the control field in the cpsr but allows read-write access to the conditional flags.
- There are seven processor modes: six privileged modes and one privileged mode.
- The privilege modes are abort, fast interrupt request, interrupt request, supervisor, system & undefined. The non-privileged mode is user.

1. The processor enter abort mode when there is a failure to attempt to access memory.
2. Fast Interrupt request and Interrupt request modes correspond two interrupt levels available on the ARM processor.
3. Supervisior (System) mode is a special version of a user mode that allows full read-write access to the cpsr.
4. System mode is a supervisior mode is the mode that the processor is in after rest & is generally the mode that an OS kernal operators in.
5. Undefined mode is used when the processor encounters an instruction

that is undefined or not supported by the implementation.
- User mode is used for program and applications.

13] Explain the concept of exceptions, interrupts and the vector table.
→ • When an exception or interrupt occurs, the processor sets the program counter (PC) to a specific memory address.
- The address is within a specific address range called the vector table.
- The entries in the vector table are the instructions that branch to specific routines designed to handle particular exception or interrupt.
- The memory map address 0x00000000 is reserved for the vector table, a set of 32-bit words.
- On some processors, the vector table ~~are the~~ can optionally located at higher address in memory starting at the 0xffff0000.
- When an exception or interrupt occurs, the processor suspends normal execution and starts loading instructions from the exception vector table.
- Each vector table entry contains a form of branch instruction pointing to start of a specific routine.
- Following is the vector table:

Exception/Interrupt	Shorthand	Address
Reset	RESET	0x00000000
Undefined instruction	UNDEF	0x00000004
Software interrupt	SWI	0x00000008
Prefetch abort	PABT	0x0000000C
Data abort	DABT	0x00000010
Reserved	--	0x00000014
Interrupt request	IRQ	0x00000018
Fast Interrupt request	FIQ	0x0000001C

- Rest vector is the location of the first instruction executed by

the processor when power is applied.
- Undefined instruction vector is used when the processor cannot decode the instruction.
- Software interrupt vector is called when SWI instruction is executed.
- Prefetch abort vector occurs when the processor attempts to fetch an instruction attempts to access data memory without the correct access permissions.
- Data abort vectors is similar to prefetch abort but is raised when an instruction attempts to a access data memory without the correct access permissions.
- Interrupt request vector is used by external hardware to interrupt the normal execution flow of the processor.
- Fast interrupt request vector is similar to the interrupt request but is reserved for hardware requiring faster response times.

14] Explain the arithmetic instructions with respect to the ARM processor.

→ • The Arithmetic instructions implement addition and subtraction of 32-bit signed and unsigned values.

syntax : <Instruction>{<cond>}{S} Rd, Rn, N

ADC	add two 32-bit values and carry	Rd = Rn + N + carry
ADD	add two 32-bit values	Rd = Rn + N
RSB	reverse subtract of two 32-bit values	Rd = N - Rn
RSC	reverse subtract with carry of 2 32-bit values	Rd = N - Rn - !(carry flag)
SBC	subtract with carry of two 32-bit values	Rd = Rn - N - !(carry flag)
SUB	subtract two 32-bit values	Rd = Rn - N

- N is the result of the shifter operation. The syntax of shifter operation is shown above table.

D D M M Y Y Y Y

Ex: The simple abstract instruction subtracts a value stored in register r2 from a value stored in register r1. The result is stored in register r0.

PRE r0 = 0x00000000, r1 = 0x00000009, r2 = 0x00000001

SUB r0, r1, r2

POST r0 = 0x00000001

5) With example. Discuss the following ARM instructions.

i) MOV ii) MVN iii) ADD iv) SUB v) ORR

→ i) MOV

> Move a 32-bit value into a register

Rd = N

Ex: The MOV instruction takes the content of register r5 and copies them into register r7, in this case, taking the value 5 and overwriting the value 8 in register r7

i) PRE r5 = 5, r7 = 8

MOV r7, r5

POST r5 = 5, r7 = 5

ii) MVN

> Move the NOT of the 32-bit value into a register

Rd = NN

Ex: The MOV instruction takes the content of register r5 and copies them into register r7, in this case taking the value 5 and overwriting the value 8 in register r7

ii) PRE r5 = 5, r7 = 8

MVN r7, r5 ; r7 = Nr5

POST r5 = 5, r7 = 10

iii) ADD

> Add two 32-bit values

Rd = Rn + N

Ex: This simple addition instruction adds a value stored in register r2 from a value stored in register r1. The result is stored in register r0.

PRE r0 = 0x00000000 r1 = 0x00000004 r2 = 0x00000002

ADD r0, r1, r2

POST r0 = 0x00000006

iv) SUB

> subtract two 32-bit values

Rd = Rn - N

Ex: This simple subtraction subt instruction subtracts a value stored in register r2 from a value stored in register r1. The result is stored in register r0.

PRE r0 = 0x00000000 r1 = 0x00000002 r2 = 0x00000001

SUB r0, r1, r2

POST r0 = 0x00000001

v) ORR

> logical bitwise OR of two 32-bit values

Rd = Rn | N

Ex: This example shows a logical OR operation b/w registers r1 & r2. r0 holds the result.

PRE r0 = 0x00000000, r1 = 0x02040608, r2 = 0x10305070

ORR r0, r1, r2

POST r0 = 0x12345678.

D D M M Y Y Y Y

18) Describe ARM assembly language program to perform addition and multiplication of two 32-bit numbers.

→ Addition of two 32-bit Numbers.

Program:

```
Assembly
AREA ADD32, CODE, READONLY
ENTRY
LDR R1, =NUM1   ; Load address of first number
LDR R2, [R1]    ; R2 = NUM1
LDR R1, =NUM2   ; Load address of second number
LDR R3, [R1]    ; R3 = NUM2
ADDS R4, R2, R3 ; R4 = R2 + R3
LDR R1, =RESULT
STR R4, [R1]    ; Store Result
STOP B STOP
NUM1 DCD 0x12345678
NUM2 DCD 0x11111111
RESULT DCD 0
END
```

Multiplication of two 32-bit Numbers

```
Assembly
AREA MUL32, CODE, READONLY
ENTRY
LDR R1, =NUM1
LDR R2, [R1]    ; R2 = NUM1
LDR R1, =NUM2
LDR R3, [R2]    ; R3 = NUM2
MUL R4, R2, R3  ; R4 = R2 x R3
LDR R1, =RESULT
STR R4, [R1]    ; store result
STOP B STOP
NUM1 DCD 0x00000005
NUM2 DCD 0x00000004
RESULT DCD 0
END
```

∴ continue →

19] Illustrate program for forward and backward branch by considering an example.

→ B forward
ADD r1, r2, #04
ADD r0, r6, #02
ADD r3, r7, #4
forward
SUB r1, r2, #4

The forward branch skips three instructions.

0x00000000
0x00000004
0x00000008
0x0000000C
0x00000010

Backward

Add r1, r2 #4
Sub r1, r2 #4
Add r4, r6, r7
Backward

0x00000014
0x00000018
0x0000001C
0x00000020

D D M M Y Y Y Y

22] Explain the ARM swap instruction with an example code.

→ • The swap instruction is a special case of a load-store instruction. It swaps the contents of memory with the contents of a register.

Syntax : SWP {B} {<cond>} Rd, Rm [Rn]

SWP	swap a word between memory and a register.	tmp = MEM32 [Rn] MEM32 [Rn] = Rm Rd = tmp
SWPB	swap a byte between memory and a register.	tmp = MEM8 [Rn] MEM8 [Rn] = Rm Rd = tmp

PRE MEM32 [0x9000] = 0x12345678
r0 = 0x00000000
r1 = 0x11112222
r2 = 0x00009000

POST MEM32 [0x9000] = 0x11112222
r0 = 0x12345678
r1 = 0x11112222
r2 = 0x00009000

24] Explain Barrel shifter with a neat labelled diagram.

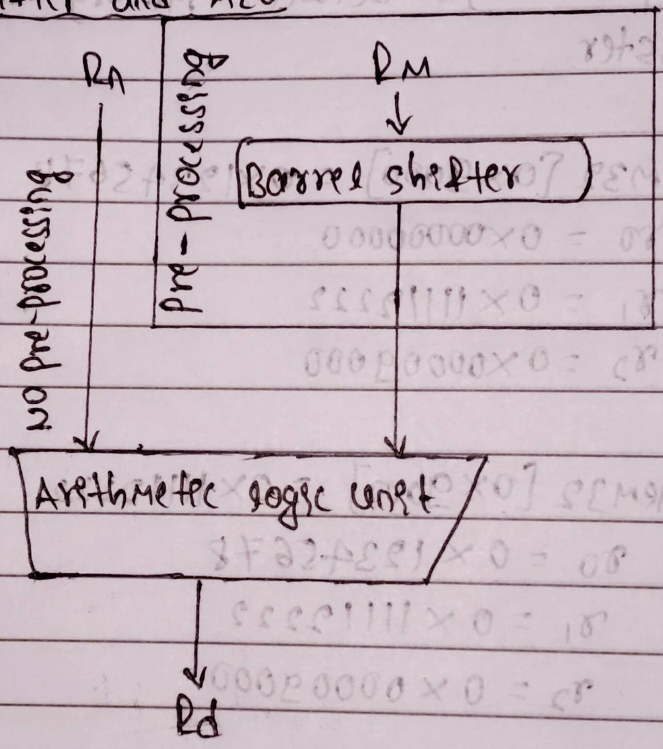
→ • A unique and powerful feature of the ARM processor is the ability to shift the 32-bit binary pattern in one of source registers left or right by a specific number of positions before it enters the ALU.
• This shift increases the power and flexibility of many data processing operations.

- For example, we apply a logical shift left (LSL) to register Rm before moving it to the destination register.

```
PRE
  r5=7
  r7=8
  MOV r7, r5, LSL #2
POST r5=5
  r7=20
```

- The above example shift logical left r5 = 5 (00000101 in binary) by two bits and then r7 = 20 (00010100 in binary).

Fig: Barrel shifter and ALU



Barrel shifter operation

Mnemonic	Description	Shift	Result	Shift amount
LSL	logical shift left	x LSL y	x << y	# 0-31 or Rs
LSR	logical shift right	x LSR y	(unsigned) x >> y	# 1-32 or Rs
ASR	arithmetic right shift	x ASR y	(signed) x >> y	# 1-32 or Rs
ROR	rotate right	x ROR y	((unsigned) x >> y) | (x << 32-y))	# 1-31 or Rs
RRX	rotate right extended	x RRX y	(c flag << 31) | ((unsigned) x >> 1)	none

D D M M Y Y Y Y

Note: x represents the register being shifted any y represents the shift amount

Barrel shift operation syntax for data processing instructions.

N shift operations	syntax
Immediate	# immediate
Register	Rm
Logical shift left by immediate	Rm, LSL # shift_imm
Logical shift left by register	Rm, LSL Rs
Logical shift right by immediate	Rm, LSR # shift_imm
Logical shift right with register	Rm, LSR Rs
Arithmetic shift right by immediate	Rm, ASR # shift_imm
Arithmetic shift right by register	Rm, ASR Rs
Rotate right by immediate	Rm, ROR # shift_imm
Rotate right by register	Rm, ROR Rs
Rotate right with extend	Rm, RRX